AI 에이전트 개발 — 발표자 대본

- 슬라이드 수: 40

- 부제: LangChain · LangGraph · Deep Agents 미리보기

- 기준: source-repo/01_beginner/*.ipynb

---

01. AI 에이전트 개발

안녕하세요. AI 에이전트 개발 과정에 오신 걸 환영합니다. 이 과정에서는 LangChain, LangGraph, Deep Agents 세 가지로 LLM 에이전트를 처음부터 만들어 봅니다. Python 기초만 있으면 충분해요.

---

02. 오리엔테이션

먼저 큰 그림부터 보겠습니다. 세 프레임워크가 어떻게 한 계열로 이어지는지, 이 과정이 어디로 향하는지 짚고 가겠습니다.

---

03. 세 프레임워크는 경쟁이 아니라 한 계열

헷갈리기 쉬운데, 셋은 경쟁 관계가 아닙니다. 맨 아래 LangChain이 모델과 도구를 잇는 토대이고, 그 위에 LangGraph가 흐름 제어를 얹습니다. Deep Agents는 그 위에 파일, 서브에이전트, 백엔드가 붙은 장기 작업 하네스입니다. 계획 도구는 이제 기본이라고 외우기보다, 필요한 작업에서 TodoListMiddleware로 붙인다고 기억하면 됩니다.

---

04. 과정 로드맵 — 노트북 하나씩 쌓아 올립니다

진행 순서입니다. LLM 기초로 시작해서 LangChain으로 첫 에이전트를 만들고, 메모리를 붙이고, LangGraph로 흐름을 직접 그려보고, Deep Agents를 거쳐 마지막에 검색·요약 에이전트로 종합합니다. 각 단계마다 노트북을 직접 돌려볼 거예요.

---

05. 이 과정에서 직접 만드는 것

이 과정의 결과물은 분명합니다. 질문을 주면 웹을 검색하고 한국어로 요약해 주는 리서치 에이전트를 마지막에 직접 완성합니다. 개념만 배우고 끝나지 않고, 실제로 도는 에이전트가 손에 남아요.

---

06. 환경 설정

그럼 손부터 풀어보겠습니다. 키를 넣고 모델을 띄워서 첫 응답을 받는 데까지 해볼게요.

---

07. 에이전트란 무엇인가

에이전트가 뭔지부터 짚고 가죠. 그냥 LLM은 물으면 한 번 답하고 끝이지만, 에이전트는 어떤 도구를 쓸지 스스로 판단하고, 실행하고, 결과를 보고 다음 행동을 정하는 걸 반복합니다. 추론하고 행동하는 거예요.

---

08. 키 발급과 환경 변수 로드

준비물은 단출합니다. OpenAI 키는 필수고, 검색용 Tavily 키는 마지막 프로젝트에서만 필요해요. 키는 .env 파일에 두고 깃에는 절대 올리지 않습니다. ▶ 화면 오른쪽처럼 .env에 키를 적고, load_dotenv로 불러온 뒤 잘 들어왔는지 확인하면 됩니다.

09. ChatModel로 LLM을 정의합니다

이제 모델을 정의합니다. ChatOpenAI는 OpenAI 모델을 LangChain에서 쓰기 좋게 감싼 클래스예요. 여기서 만든 이 model 객체를 앞으로 계속 재사용합니다. init_chat_model을 쓰면 이름만으로 프로바이더를 자동 감지해서, 나중에 모델을 바꿔 끼울 때 편합니다.

10. model.invoke()는 문자열이 아니라 AIMessage를 돌려줍니다

한 가지 짚을 게, model.invoke의 결과는 그냥 문자열이 아니라 AIMessage 객체입니다. .content에 답이 들어있고, .usage_metadata에 토큰 수, .tool_calls에 도구 호출 정보까지 함께 담겨요. 그래서 응답뿐 아니라 비용·도구 사용까지 한 객체로 볼 수 있습니다.

11. 관측성 — 어디서 실패했고 토큰은 얼마나 썼나

에이전트는 여러 단계를 거치기 때문에, 어디서 실패했고 토큰을 얼마나 썼는지 보려면 트레이싱이 사실상 필수입니다. LangSmith는 환경변수 한 줄로 자동으로 켜지고, Langfuse는 오픈소스 대안이에요. 벤더에 안 묶이고 싶으면 OpenTelemetry로 Datadog이나 Grafana 같은 데로 보낼 수도 있습니다.

12. LLM 기초

에이전트로 가기 전에, 그 재료인 LLM을 제대로 다루는 법부터 익히겠습니다. 메시지, 프롬프트, 스트리밍이에요.

13. LangChain 메시지 타입 — System · Human · AI · Tool

LLM에 말을 거는 기본 단위는 메시지 리스트입니다. System은 행동 지침, Human은 사용자 입력, AI는 모델 응답이고요. 도구 결과를 돌려줄 땐 ToolMessage를 씁니다. 스트리밍 중에 나오는 조각은 AIMessageChunk인데, 합쳐지면 최종 AIMessage가 돼요.

14. 시스템 메시지가 모델의 성격을 결정합니다

여기서 가장 강력한 게 시스템 메시지예요. 같은 질문이라도 시스템 메시지를 바꾸면 답의 톤과 관점이 완전히 달라집니다. 이게 프롬프트 엔지니어링의 출발점이고요. 메시지는 객체로 줘도 되고 딕셔너리로 줘도 결과는 같습니다.

15. 호출은 세 가지 — invoke · stream · batch

호출 방식은 세 가지를 기억하세요. invoke는 전체 응답을 한 번에, stream은 토큰이 나오는 대로 실시간으로, batch는 여러 입력을 병렬로 처리합니다. ▶ stream을 돌려보면 글자가 주르륵 흘러나오는데, 사용자를 기다리게 하지 않는 이 느낌이 UX 포인트예요.

16. 첫 에이전트 — LangChain

이제 진짜 에이전트를 만들어 봅니다. LangChain으로 가장 빠르게요.

17. ReAct 루프 — 생각하고 행동하기를 반복합니다

LangChain 에이전트는 ReAct 루프로 돕니다. 사용자 입력이 들어오면 모델이 도구를 쓸지 판단하고, 필요하면 도구를 실행해서 결과를 받고, 그걸 보고 다시 판단하는 걸 반복해요. 답이 나오거나 최대 횟수에 도달하면 멈춥니다.

18. 도구 만들기 — docstring이 가장 중요합니다

도구 만드는 법은 의외로 쉬워요. 평범한 함수에 @tool만 붙이면 됩니다. 타입 힌트가 입력 형식을 정하고, docstring이 이 도구가 뭐 하는 건지를 모델에게 알려줘요. 그래서 설명을 잘 써주는 게 정말 중요합니다.

19. create_agent — 모델·도구·프롬프트를 묶습니다

이제 모델·도구·시스템 프롬프트를 create_agent에 묶으면 끝이에요. 입력은 messages 리스트로 주고, 답은 마지막 메시지에서 꺼냅니다. ▶ '15 더하기 27' 하고 물으면 에이전트가 알아서 add 도구를 골라 쓰고 답을 줍니다.

20. 멀티턴 메모리

그런데 방금 만든 에이전트엔 약점이 하나 있어요. 대화를 기억하지 못합니다. 이걸 해결해 보겠습니다.

21. 메모리 없는 에이전트의 한계

기본 에이전트는 매번 새 대화로 취급해요. '내 이름은 철수야' 다음에 '내 이름이 뭐야' 하고 물으면 답을 못 합니다. invoke가 처리하고 나면 아무것도 저장하지 않기 때문이에요.

22. checkpointer와 thread_id로 대화를 기억합니다

해결책은 checkpointer입니다. InMemorySaver를 붙이고 thread_id로 '같은 대화'라고 표시해 주면 돼요. ▶ thread_id를 똑같이 두고 두 번 물어보면, '그 결과에 2를 곱해'라는 말을 알아듣습니다. 메모리가 붙었다는 증거예요.

23. 단기에서 영속으로, 그리고 긴 대화 관리

InMemorySaver는 프로세스가 꺼지면 사라지니까, 실제 서비스에선 PostgresSaver 같은 DB 체크포인터로 바꿉니다. API는 똑같아서 한 줄만 교체하면 돼요. 그리고 대화가 길어지면 트리밍·요약·삭제로 토큰을 관리합니다.

24. LangGraph 워크플로

LangChain이 흐름을 알아서 감춰줘서 편했다면, 이번엔 그 흐름을 직접 그려보겠습니다. LangGraph예요.

25. 왜 create_agent를 넘어가나

실제 앱은 조건 분기, 병렬 실행, 사람 승인, 오류 복구 같은 게 필요합니다. 단순 ReAct 루프만으로는 부족해요. LangGraph는 이걸 방향 그래프로 그려서, 중단했다 재개할 수 있는 내구적인 실행을 만들어 줍니다.

26. 구성요소는 셋 — 상태 · 노드 · 엣지

구성요소는 셋입니다. State는 노드들이 공유하는 데이터, Node는 상태를 받아 일을 하는 함수, Edge는 다음에 어디로 갈지 정하는 연결이에요. 노드는 무엇을 한다, 엣지는 다음에 어디로 간다, 이렇게 기억하시면 됩니다.

27. 기본 흐름 — 정의에서 실행까지 다섯 단계

기본 흐름은 다섯 단계예요. 그래프를 만들고, 노드를 더하고, 엣지로 잇고, 컴파일하고, 실행. 컴파일할 때 빠진 연결이나 도달 못하는 노드 같은 오류를 잡아줍니다. 노드는 전체 상태가 아니라 바뀐 부분만 돌려주면 돼요.

28. LLM을 노드로 — MessagesState와 리듀서

이번엔 노드 안에서 LLM을 불러봅니다. MessagesState를 쓰면 메시지를 담는 칸과 누적해 주는 리듀서가 이미 들어있어요. ▶ 이 chatbot 그래프를 돌려보면, 아까 create_agent가 한 줄로 감춰뒀던 흐름을 우리가 직접 그려본 게 됩니다.

29. Deep Agents 올인원

직접 그리는 것도 좋지만, 파일 작업과 서브에이전트가 필요한 장기 작업에는 Deep Agents 하네스가 편합니다. 계획이 필요한 작업에서는 TodoListMiddleware를 명시적으로 붙입니다.

30. 하네스의 기본 기능과 선택 미들웨어

Deep Agents는 하네스 자체와 선택 미들웨어를 구분해서 봐야 합니다. 파일 작업, 백엔드, 서브에이전트는 장기 작업의 중심 표면이고, 단계 계획은 TodoListMiddleware를 붙였을 때 활성화됩니다. 그래서 최신 버전에서는 모델만 넘기면 계획까지 항상 켜진다고 설명하지 않습니다.

31. create_deep_agent — 도구는 순수 함수로 넘깁니다

도구를 순수 함수로 넘기는 패턴은 유지됩니다. 달라진 포인트는 계획입니다. 리서치나 보고서처럼 여러 단계를 추적해야 할 때 TodoListMiddleware를 middleware 목록에 명시합니다. 이 한 줄이 있어야 todo 상태를 수업에서 다룰 수 있습니다.

32. 세 프레임워크 비교

셋을 다 만들어봤으니, 이제 머릿속에서 정리할 시간입니다. 언제 뭘 쓸지요.

33. 프레임워크 특성 한눈 비교

추상화 수준은 앞 1장에서 봤으니, 여기선 실무 특성으로 가르겠습니다. 모델 지원은 LangChain만 100개가 넘는 프로바이더를 직접 쓰고 나머지는 그걸 공유해요. 상태 관리는 LangGraph와 Deep Agents가 체크포인터를 쓰고, 샌드박스 실행은 Deep Agents가 지원합니다. 라이선스는 셋 다 MIT예요.

34. 이럴 때 씁니다 — 의사결정 기준

선택 기준은 '어디까지 직접 제어하고 싶은가'예요. 간단한 도구 호출이면 LangChain, 분기와 사람 개입이 있는 워크플로면 LangGraph, 파일·계획이 필요한 장기 작업이면 Deep Agents. 반대로 그 수준이 필요 없으면 과한 선택이 됩니다.

35. 같은 질문, 세 가지 방식

같은 '3 더하기 4' 질문을 세 방식으로 짜보면 차이가 한눈에 들어옵니다. LangChain은 한 줄, LangGraph는 흐름을 직접 그리고, Deep Agents는 하네스가 다 들어있죠. 하나만 평생 쓰는 게 아니라 작업에 맞춰 고르고 섞는 겁니다.

36. 미니 프로젝트

자, 배운 걸 다 모아서 진짜 하나 만들어 봅시다. 검색하고 요약하는 리서치 에이전트예요.

37. 검색 도구 정의 — Tavily

먼저 검색 도구부터요. Tavily 클라이언트를 함수로 감싸고, docstring과 타입 힌트, Args 설명까지 적어줍니다. Literal로 허용 값을 제한하고 기본값을 두면, 에이전트가 이 도구를 정확히 쓸 수 있어요. Tavily 키는 무료로 발급받으시면 됩니다.

38. Deep Agents 리서치 에이전트

검색 도구와 시스템 프롬프트만으로도 에이전트는 만들 수 있습니다. 다만 긴 리서치처럼 단계 관리가 필요하면 TodoListMiddleware를 함께 붙여야 합니다. 이렇게 하면 검색, 중간 메모, 종합 단계를 todo로 관찰하면서 설명할 수 있습니다.

39. LangChain 버전과 비교, 그리고 다음 단계

같은 걸 LangChain으로도 만들어 비교해 보면 성격 차이가 분명해져요. LangChain은 바로 검색하고 요약하는 데 가볍고, Deep Agents는 파일 작업과 서브에이전트가 필요한 장기 작업에 맞습니다. todo 계획은 Deep Agents 0.7 기준으로 명시한 미들웨어가 담당합니다.

40. 작은 조각들이 하나로 이어졌습니다

수고하셨습니다. LLM 한 번의 호출에서 출발해서, 도구를 붙이고, 에이전트를 만들고, 흐름을 그려보고, 마지막엔 직접 리서치 에이전트까지 완성했어요. 작은 조각들이 같은 인터페이스 위에서 하나로 이어진 겁니다.